

CPT202

Final Report for Software Engineering Group Project

2022/2023 Semester 2

<Project Title> Pet Shop Appointment System

<Date> 2023/5/19

Group number: <Group 10>

Student Name: Leduo Chen

Student ID: 2035836

Project URL: <http://123.57.217.243:8080/appointment-system>

Table of Content

Introduction	3
Software Development Process	3
Software Design	3
Change Management	11
Legal, social, ethical, and professional issues	15
Conclusion	15
Appendix	16

Introduction

This report encapsulates my personal journey of developing a web-based pet grooming appointment system for a pet shop with my team. This system empowers customers to book grooming appointments for their pets in advance, ensuring a seamless service experience. Our team successfully delivered a solution encompassing crucial features like user registration, account management, appointment booking and management, grooming service pricing, sales strategies, master file maintenance, and business analytics. My personal involvement was mainly centered around user registration and account management, where I designed and implemented functionalities like user registration, login, logout, and security mechanisms for these operations. I also spearheaded the feature of adding pets during the order process and the security interceptor design for our website.

From my perspective, our solution effectively caters to the client's needs, offering an easy-to-use, secure platform for customers to book their pet grooming services effortlessly. The strengths of our system lie in its comprehensive functionality, user-friendly interface, and robust security measures. However, certain weaknesses could be addressed in future iterations. For instance, our system's scalability may need to be improved when handling many user requests. Also, the accuracy and fairness of our groomer rating system might benefit from more sophisticated feedback mechanisms. In future enhancements, consider augmenting our system's scalability to handle more user requests and introducing a more complex groomer rating mechanism, incorporating customer reviews and feedback.

In this report, we succinctly discuss the crucial aspects of our project, including the Scrum-driven software development process, individual software design, effective change management, and considerations of legal, social, ethical, and professional issues within the context of our project. Mainly focus on my personal contribution to the group and my thoughts on this project

Software Development Process

In this project, we chose Scrum as the software development framework. Since our team consists of nine members, if an appropriate development framework is not adopted, team members may develop according to their ideas and programming habits, making it difficult to form a coherent and complete project. The agile nature of Scrum allows us to control the project schedule better and maintain good communication between teams.

Our development process is divided into three Sprints, each forming a complete development cycle, covering the prioritization, design, development, and testing of the PBI. Each Sprint goes through a Sprint Plan, Process, and Review. The following is a detailed description of our team's Scrum development process:

Sprint 0: In the preparation and planning phase we did some essential preparation work in this phase. Firstly, we set up the team's synchronous development platform on Github and started using Git for version control and code update. Secondly, we defined and configured a

unified development environment (Java 16.0.2, MySql 8.0.31) to ensure that all team members were developing in the same environment and to avoid problems caused by environmental differences. Finally, we have analyzed the project requirements in depth, extracted the PBI list from Assignment 1, and ranked it by priority to plan for subsequent development work.

Sprint 1: In the first Sprint, we focused on implementing the highest-priority PBIs. We followed the Scrum requirements and had daily stand-up meetings to discuss everyone's work progress, identify and resolve issues and ensure the team was in sync. Following the division of labor, I was mainly responsible for implementing the core functions such as user login registration and account management. During this phase, we were also introduced to the issue of requirement changes and learned how to handle these changes within the Scrum framework. For example, the PBIs of high-priority requirement changes are immediately added to this Sprint, and the requirements with less high requirements are left to the next Sprint to complete. Later part of Change Management will discuss in detail how we deal with the changed requirements.

Sprint 2: In the second Sprint, we started to work on slightly lower priority but still important PBIs, including new requirements identified in Sprint 1. For example, I added a security blocker in this phase to prevent non-logged-in users from accessing protected URLs, a requirement that was discussed during the development of Sprint 1. In addition, I was also responsible for completing the function to add pets and select them on the appointment placement screen, a requirement of such high importance that it was identified and immediately added to Sprint 2. We followed the Scrum development model, maintaining efficient communication and collaboration to ensure smooth development.

Sprint 3: In the last Sprint, we dealt with the remaining PBIs, mainly ancillary features and optimization items. For example, I focused on front-end design optimizations and numerous JavaScript-based front-end form validations to prevent error messages from being uploaded to the back end, causing errors. In addition, we also optimized and tweaked features developed in the first two Sprints to improve the performance and user experience of the system.

Scrum ensures the smooth progress of our work in terms of development content and helps us discover and solve problems in teamwork in time. For example, at the beginning of the project, we were not used to using GitHub, nor did we develop the habit of updating the code repository in time after each task was completed. This makes it difficult for us to find the problem from the accumulated unsynchronized code if there is a conflict or bug when pushing or pulling the code, resulting in the entire warehouse becoming unavailable. However, thanks to the Scrum framework's daily stand-up meetings, we were able to spot soon and fix this problem quickly. This allows our follow-up development process to proceed smoothly and significantly improve efficiency.

In addition, the daily meeting routines and meeting check-ins have significantly increased the motivation of the team members and prevented delays in the development process.

The above shows that the Scrum framework plays a key role in our software development process. It allows us to respond quickly to requirements changes and ensures effective communication between teams.

Software Design

In this section, I will explain in detail the design and implementation process of each PBI that I developed independently in the project. Each PBI will be carried out according to the design and implementation process, and the UML diagram I drew during the design process will also be provided to assist in understanding. However, some functional details are discovered during the actual development process, so even the logic of design and implementation are completely consistent but the details are not exactly the same. Throughout the development process, I chose to use the Spring MVC framework and Thymeleaf template engine, combined with the MVC (Model-View-Controller) design pattern, to ensure the readability and modularity of the code. The front-end part uses CSS and JavaScript for design and function realization.

1. PBI: Pop-up Window Design

User Story: As a user, I desire a Pop-up window component to replace some page transitions, enabling a more seamless and cohesive user experience.

Requirement Analysis:

Aiming to provide a smoother and more cohesive user experience, we wish to minimize the number of page transitions. Every page transition could disrupt user orientation within the application, leading to a break in the user experience. Moreover, page transitions may lead to unnecessary server requests and responses, adding to network load. Thus, we have chosen to use a Pop-up window component to substitute some page transitions. The login, registration and adding pets mentioned later all use the content of this pbi, so it is placed first.

Design:

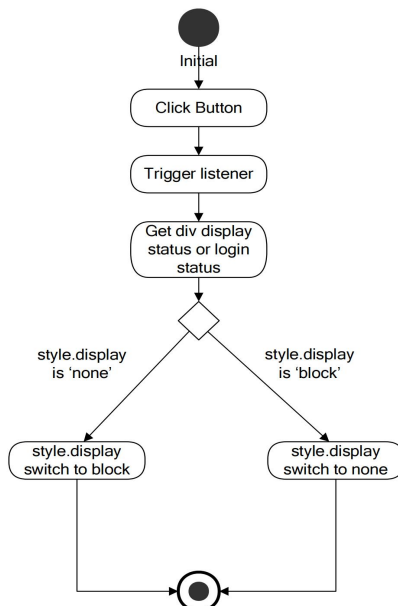
According to Figure x, We designed a Pop-up window component to display related content when the user clicks a specific button. The default style of this modal window is set to be hidden (style set to "display:none"). Furthermore, we added event listeners to each button that should trigger the modal, ready to respond to user clicks.

Implementation:

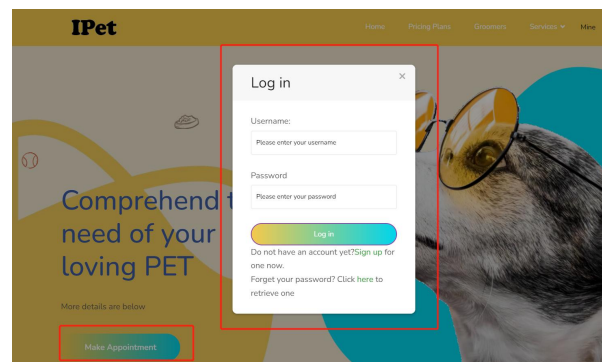
Upon the user clicking a specific button, our event listener is triggered. In this event listener, we first get the current display state of the modal window (style.display attribute) and then switch the visibility of the modal based on the current state. If the modal is currently visible (style.display is "block"), we hide it (set style.display to "none"); if the modal is hidden (style.display is "none"), we display it (set style.display to "block"). For some buttons, additional checks for login status are performed to toggle the bound method. For example, when a unlogged user clicks make appointment button, the login interface will pop up, while a

logged-in user clicks make appointment button to jump to the order interface.

Through this approach, we are able to display or hide specific content in response to user actions without navigating away from the page. This design makes our application interface smoother, and users can complete more actions within a single page, reducing the need for frequent page transitions.



Figure[1]. Design for pop-up window.



Figure[2]. Implementation of pop-up window

2. PBI: Adding New Pets in the Appointment Booking Page

User Story: As a customer, I want the ability to add a new pet to my account directly from the appointment booking page, preventing the need for navigating to different pages to make an appointment.

Design:

Initially, we verified the business logic for the process of adding a new pet. It necessitates that the user should be able to add pet information, such as the pet's name, size, and type. And the form should be the pop-up like PBI 1 shows.

Implementation:

Service and Model Layer:

In the service layer, we implemented an 'addPet' method that accepts a Pet model object and a MultipartFile as parameters. The method prepares a unique image path for the pet's picture using the provided MultipartFile, and then saves the Pet object in the pet Repository. The method then returns a status code based on the result.

Controller Layer:

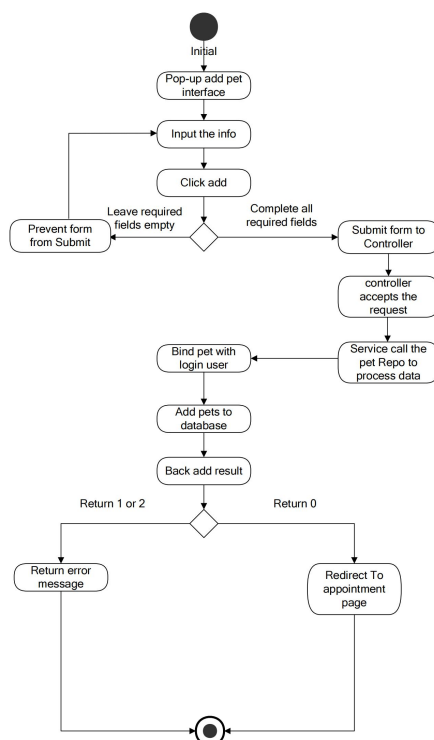
In the controller layer, we developed a 'addPetPost2' method that processes the POST request.

The method takes HttpSession, Model, a Pet model object, and MultipartFile as inputs. The logged-in user's username is extracted from the session and used to retrieve the associated User object. This User object is then set into the Pet model, establishing an association between the user and the pet. The method then calls the `addPet` function from the service layer, which returns a status code that is added to our model to display appropriate messages on our view. Depending on the status code, the user is either redirected to the 'Appointment' view or remains on the 'MyPet8' view.

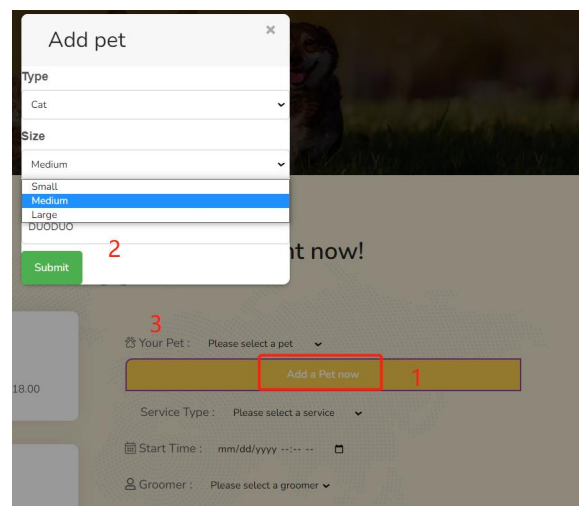
View Layer:

In the view layer, we used a pop-up form, created through PBI pop-up UI, for users to enter their pet's details. We leveraged JavaScript for front-end validation of the form, ensuring that users fill all necessary fields before submitting.

Like figure[x] shows, After user add their own pet to the database and redirect back to the make appointment interface, they can select the pet in the pet selection. And they can directly select the added pet when making appointment next time. This function allows user adding pets when placing an order without unnecessary page jumps, making the user experience more streamlined and comfortable.



Figure[3]. Design for add pet



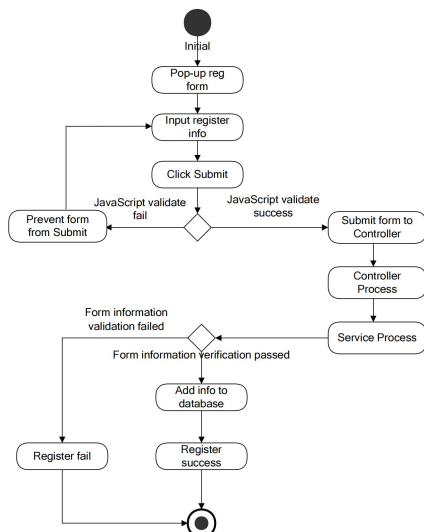
Figure[4]. Add pet in the make appointment page.

3. PBI: Registration

User Story: As a new user, I want to be able to register an account by providing my personal information and creating login credentials so that I can become a registered user and access the system.

Design:

In our registration function, we employ a multi-layer authentication strategy from front-end to back-end. During the design process, we first determined the business logic of registration. Users need to provide some basic information when registering, such as user name, password, phone number and email. We need to check in the database if the username and email is registered, if not, we save the new user's information into the database and automatically log in for the user. The activity diagram of the design is shown below.



Figure[5]. Design for register.

Implementation:

View layer:

In the view layer, I use JavaScript to implement the form verification function to verify the information entered by the user in real time. If the input does not meet the requirements, our JavaScript validation code provides feedback when the user tries to submit the form and prevents the form from submitting. However, although front-end validation can provide immediate feedback and a good user experience, it does not guarantee complete security of data, so we also validate at the service and controller layers.

The screenshot shows a 'Register' form with the following fields and errors:

- Username:** Personal-Report (no error)
- Email:** Leduo.Chen20@student (Error: Email format is incorrect, please re-enter)
- Phone Number:** Please enter your Phone Number (Error: Phone number can not be empty)

The screenshot shows a password form with the following fields and errors:

- Password:** (masked with dots, no error)
- Repeat password:** (masked with dots, Error: The passwords entered twice are inconsistent, please re-enter)

Buttons: Submit, Already have an account? Log in here.

Figure[6]. Effect of front form validation.

Controller layer:

In the controller layer, we created a registerUser method that handles POST requests. This method receives the information entered by the user as a parameter, and then calls the registerUser method of the service layer. Depending on the method's return value, it sets the appropriate error or success message. Finally, it redirects to the corresponding URL, according to which the view will be generated.

Service and Model layer:

In the service layer, we implement the registerUser method, which checks whether the username and email are already registered, and check whether the registration information meets the specifications and requirements according to the same logic as the front end. If everything is correct, it sets the user's registration time to the current time, and then saves the user's information to the database through UserRepository.

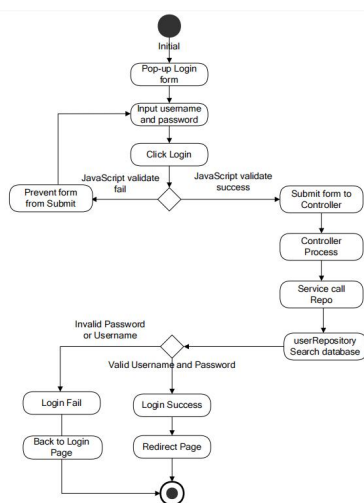
In general, our registration function adopts a multi-layer verification strategy. From the front end to the back end, we have carried out strict verification to ensure that the data entered by the user not only meets the business needs, but also ensures the security of the system .

4. PBI: Login

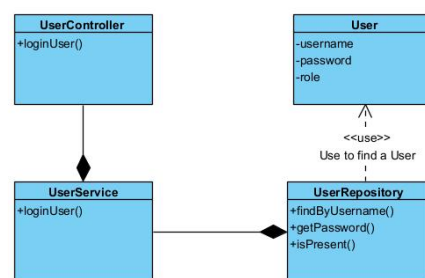
User Story: As a registered user, I want to access my account by submitting my login credentials, thereby allowing me to leverage the system's features and data tailored to my account.

Design:

Firstly, we affirmed the business logic for the login process. This necessitates a query to the corresponding user in the database when the username and password are inputted, with a subsequent check to see if the entered password aligns with that stored. If the passwords match, a relevant response is returned, contingent on the user type.



Figure[7]. Design of Login



Figure[8]. Class diagram. And all other class

relation in MVC is quite same like that.

Implementation:

In the service layer, we have instituted a login method that accepts a username and password as inputs, querying the database via the userRepository and cross-checking both the password and user type.

Controller layer

In the controller layer, we have devised a loginUser method that manages the POST request. This method takes the username and password as parameters and proceeds to call the loginUser method in the service layer. Should the password correspond with the one stored, the controller saves the User object or selected properties (such as user roles) within the session. It then communicates a success message or related data to the view for display. Lastly, it redirects to an appropriate URL, which prompts the view to generate the page.

View layer

At the view layer, we leverage the Thymeleaf templating engine to manage HTML generation. Based on the model attributes defined by the controller and the redirected URL, Thymeleaf creates an HTML page, which presents the relevant information and sends it back to the user's browser.

By adopting this approach, we have successfully executed the user login function. Our code is well-organized, with each layer tasked with specific responsibilities, thereby promoting ease of maintenance and potential future extensions.

5. PBI: Implementing an Interceptor

User Story: As a developer, I wish to implement an interceptor to control that only users with specific roles can access certain pages and resources.

Design Analysis:

In our application, certain pages and resources should only be accessed by users with specific roles. For instance, admin pages should only be accessible to admin users, and regular user pages should only be accessible to regular users. If users attempt to access pages they're not authorized for, they should be redirected to pages they can access. Additionally, certain resources (such as CSS, JS files, images, etc.) should be accessible to all users without requiring a role check. Consequently, we need to create an interceptor to fulfill these needs.

Interceptor Design:

In the Spring framework, we can utilize the HandlerInterceptor interface to implement our custom interceptor. Within our AuthenticationInterceptor interceptor, we primarily override the preHandle method, which is called before request processing. In this method, we verify the user's login status and role, and based on this information, we decide whether to allow the

request to continue or to redirect it.

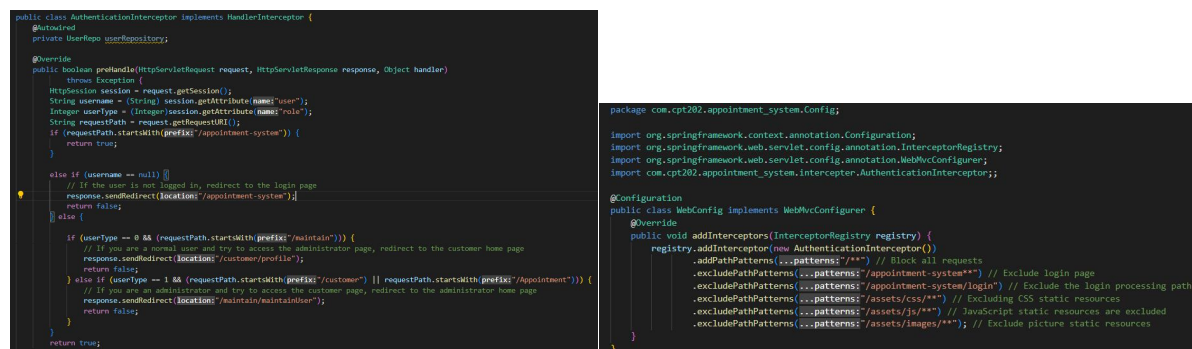
Interceptor Implementation:

In the preHandle method, we initially fetch the current session and the user's username and roles. Then, we scrutinize the request's path, if the path starts with "/appointment-system", we directly permit the request to proceed since these paths are open to all users. Next, we check the user's login status, if the user is not logged in, we redirect them to the login page. Lastly, we verify the user's role, if the user's role doesn't match the page they're trying to access, we redirect them to the homepage suited to their role.

Interceptor Registration:

In the WebConfig configuration class, we register our interceptor by overriding the addInterceptors method of the WebMvcConfigurer interface. We add the AuthenticationInterceptor to the interceptor registry and set the required interception and exclusion path patterns. All requests will be intercepted, except for the login page, paths that handle login, and paths to static resources.

Because the interceptor is not suitable to be represented by uml, the screenshot below is the core code for my implementation of the interceptor.



```
public class AuthenticationInterceptor implements HandlerInterceptor {
    @Autowired
    private UserRepository userRepository;

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler)
        throws Exception {
        HttpSession session = request.getSession();
        String username = (String) session.getAttribute("name");
        Integer userType = (Integer) session.getAttribute("role");
        String requestPath = request.getRequestURL();
        if (requestPath.startsWith(prefix) + "/appointment-system") {
            return true;
        }
        else if (username == null) {
            // If the user is not logged in, redirect to the login page
            response.sendRedirect(location) + "/appointment-system";
            return false;
        }
        else {
            if (userType == 0 && (requestPath.startsWith(prefix) + "/maintain")) {
                // If you are a normal user and try to access the administrator page, redirect to the customer home page
                response.sendRedirect(location) + "/customer/profile";
                return false;
            }
            else if (userType == 1 && (requestPath.startsWith(prefix) + "/customer") || requestPath.startsWith(prefix) + "/appointment") {
                // If you are an administrator and try to access the customer page, redirect to the administrator home page
                response.sendRedirect(location) + "/maintain/maintainuser";
                return false;
            }
        }
        return true;
    }
}

package com.cpt202.appointment_system.Config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import com.cpt202.appointment_system.interceptor.AuthenticationInterceptor;

@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new AuthenticationInterceptor())
            .addPathPatterns("**") // Block all requests
            .excludePathPatterns("**/appointment-system/**") // Exclude login page
            .excludePathPatterns("**/appointment-system/login") // Exclude the login processing path
            .excludePathPatterns("**/assets/css/**") // Excluding CSS static resources
            .excludePathPatterns("**/assets/js/**") // JavaScript static resources are excluded
            .excludePathPatterns("**/assets/images/**"); // Exclude picture static resources
    }
}
```

Figure[9]. The core code of the interceptor

The above delineates the design and implementation process of the interceptor. Through this method, we can effectively manage access to pages and resources, safeguarding our application from unauthorized access.

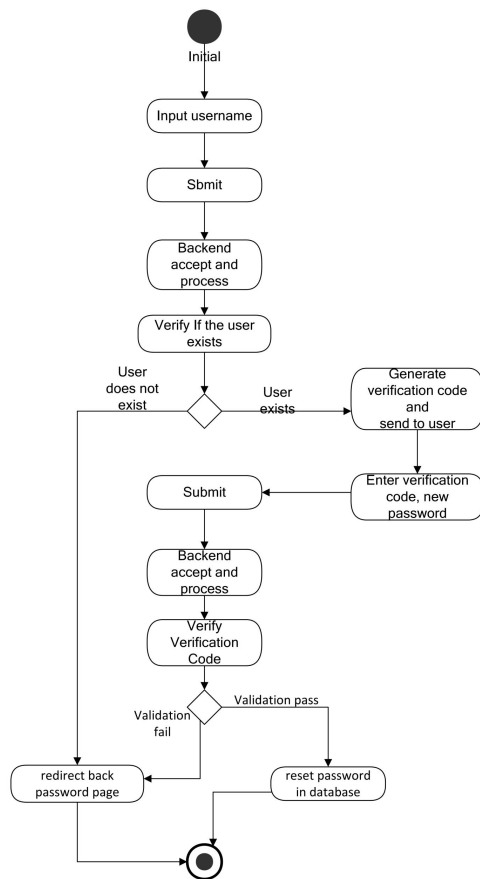
6. PBI: Password Recovery Feature

User Story: As a registered user, I would like the capability to recover or reset my password if I forget it or need to change it for security reasons.

Design:

Our business logic is shown in the figure below. Users first need to submit their username. If it exists in the database, the backend will generate and send a verification code to the email address provided by the user during registration. The user then enters the verification code and new password. If the verification code matches the verification code generated in the

background, the backend will call the model layer to modify the user's password to the uploaded new password.



Figure[10]. Design of Retrieve password.

Implementation:

Phase 1: Request verification code

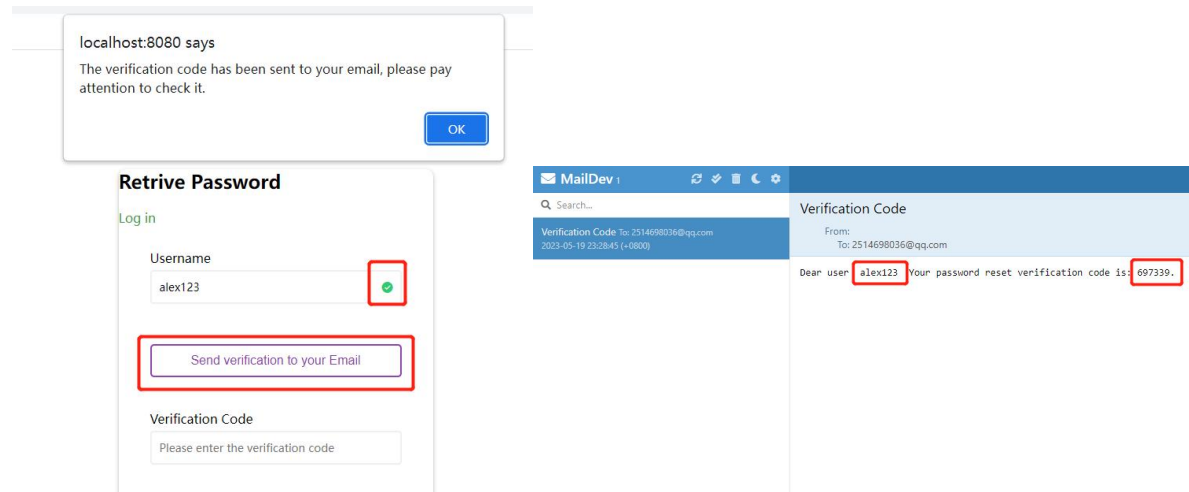
During implementation, the first step was to develop the reset password form in our view layer. When the user submits this form, it triggers a POST request containing the entered username.

The controller at the back-end is equipped with the `resetPassword` method to handle this POST request. It uses the provided username to search the user repository for a corresponding user. If no such user exists, the method responds with an error message and redirects the user back to the reset password page. However, if a user is found, the method generates a verification code and uses the injected mail service to send the code to the user's registered email. The username and verification code pair are also stored in a `HashMap` for later verification. The process is shown in figure11.

Phase 2: Verify and reset password

The user, after receiving the email with the verification code, returns to the reset password form. Here, they enter their username, the received verification code, and their new password. Upon form submission, another POST request is made, which the controller processes via

the `changePassword` method. This method verifies the entered code against the stored one in the `HashMap`. If the codes match, the method locates the user in the database, updates their password to the new one, and saves the changes. Finally, a success message is



Figure[11]. The implementation of verification code.

relayed back to the user, and they are redirected to the login page.

Through this design and implementation, we have created a robust password recovery feature that both ensures user identity verification and enhances system security.

Change Management

Managing changes is a vital facet of any project and becomes particularly essential when implementing Agile methodologies such as Scrum. Scrum is designed to accommodate and embrace change, adopting an iterative, incremental approach to maximize predictability and manage risk. It recognizes that changes are not impediments but opportunities to refine the product and deliver better value to the customer.

In our project, changes to requirements are managed as an integral part of our Scrum process. Whenever a new requirement or modification to existing requirement surfaces, we convene as a team to discuss its implications, evaluate its feasibility, and decide its priority. The new requirement is then either incorporated into the current Sprint—if it does not excessively disturb the sprint goal and work progress—or added to the product backlog to be addressed in future sprints.

Below are instances of how I use Scrum to respond and change my work to show how we've effectively managed requirement changes using Scrum. In fact, many PBI in the above design part result from change management.

1. Interceptor Requirement: During our first Sprint, we recognized the need for an interceptor to ensure more effective management of user access to specific resources. Despite the high relevance of this feature, we comprehended its complexity and importance and decided to postpone it to the subsequent Sprint. We deferred the interceptor implementation to Sprint 2 to ensure sufficient time, attention, and resources could be

devoted to it. And at that time, I was still focusing on saving the user state after login, which is the prerequisite for the interceptor. I started to develop our changing requirements in sprint2 after completing all the basic functions of login and registration in sprint1. This decision embodies Scrum's value of openness to change and the practical consideration of when and how to implement changes to prevent undue disruption to the ongoing work.

2. Front-End Validation Requirement: In the concluding phase of Sprint 1, upon re-testing the functionality developed, we identified an unexpected behavior with our registration feature. When users submitted an empty form, the back end surprisingly saved this action. This instance highlighted the necessity for front-end form validation in addition to back-end validation to prevent and correct erroneous user inputs and operations, such as reminding users when their password input does not conform to the standard rules.

In response to this discovery, we incorporated front-end validation into our project. This new requirement was considered essential for enhancing user experience and ensuring data consistency; therefore, we decided to include it in the upcoming Sprint 2. This decision reflects the principle of Scrum, which encourages responding to changes by following a fixed plan.

During the implementation in Sprint 2, we developed front-end validation for the registration feature and extended it to other forms in the application, such as preventing empty form submissions. This shows the iterative and incremental nature of Scrum, which allowed us to build on the feedback from the previous Sprint, hence enhancing the system's overall robustness.

These examples illustrate how the principles of Scrum were applied in managing changes in our project, where flexibility and adaptability in response to new insights and requirements are key. By embracing changes as opportunities to improve the product and user experience, we effectively navigated shifts in project requirements to deliver a higher-quality product.

3. Pet Size and Type Selection Requirement: During Sprint 2, we identified a need for more user experience when users were forced to navigate away from the appointment booking page to input their pet's details. As a result, we initiated a new requirement allowing users to add their pet's information directly on the booking page. Next, we discussed this new requirement in the sprint planning meeting and decided to include it in the current Sprint. I am responsible for implementing this functionality and sharing my progress and any issues faced with the team in the daily scrum meeting. In my design and implementation process, I found that to achieve this requirement, I need to implement another new pbi of designing a pop-up window on the front page. Finally, We prioritized the pop-up window design within Sprint 2 and postponed the full pet addition functionality to a future sprint. This exemplifies the Scrum practice of regularly inspecting our work and adapting our plan to continually enhance the product and user experience.

4. Enhanced Pop-up Design Requirement: As we commenced Sprint 3, we recognized the pop-up design's significant positive impact on the user experience. This acknowledgment also led us to extend the pop-up design concept to the login and registration process. This example embodies Scrum's principle of iterative development and incremental delivery,

demonstrating our flexibility and adaptability in responding to changes and opportunities for improvement.

By implementing Scrum, we managed and responded effectively to changes in our project requirements. It allowed us to adapt to new insights and feedback and improve the final product's quality and usability. This adaptability allowed us to incorporate improvements without disrupting the project flow, optimizing team productivity and ensuring the overall success of our project. Through this process, we demonstrated how Scrum's approach to change management could be effectively used to navigate the shifting landscape of project requirements.

Legal, social, ethical, and professional issues

We needed to consider various legal, social, ethical, and professional issues when developing our project. These considerations played a significant role in shaping our project and ensuring it was developed responsibly and professionally.

Certainly, I can expand on those points:

Legal Issues:

Data Privacy and Protection: Recognizing the legal and ethical implications around data privacy and protection, we adopted several measures to safeguard user information. One key strategy was storing user passwords in an encrypted format in our database. We used strong hashing algorithms to secure the passwords, which ensured that even if someone gained unauthorized access to the database, they couldn't decipher the actual passwords. This step significantly reinforced our data security and ensured compliance with laws like the General Data Protection Regulation (GDPR) and local privacy laws.

Intellectual Property Rights: We used several third-party libraries or tools to enhance our application's functionality during the development process. Aware of the legal requirements around intellectual property rights, we made sure to use only those resources that were either open-source or correctly licensed. This practice protected us from infringing on intellectual property rights, ensuring our application was built legally and ethically.

Social Issues:

User Accessibility and Inclusion: Acknowledging the social significance of digital inclusion, we were committed to making our application accessible to all users. We ensured our design adhered to universal design principles, providing an inclusive and effective user experience for diverse users. Additionally, we designed our application to be responsive, meaning it could adapt to various devices and screen sizes. This inclusive approach extended our application's reach and upheld our social responsibility.

Ethical Issues:

Respect for User Information: Our team was strongly committed to the ethical handling of user data. To ensure we respect user privacy, we strictly follow the principle of informed consent. This meant users were clearly notified about our data collection, usage, and storage policies, and we only processed their data with their explicit consent. We also reinforced user privacy by implementing a feature that verifies the user's identity before allowing password recovery.

Professional Issues:

Quality Assurance: Upholding our professional responsibility, we were committed to delivering a high-quality, reliable application. This commitment involved rigorous testing to uncover and rectify any issues. For instance, during the testing phase of Sprint 1, we discovered a need for front-end validation in the registration feature and promptly addressed it.

Responsiveness to Changes: Consistent with professional best practices in Agile development, we maintained flexibility and openness to changes throughout the project. This mindset allowed us to adapt to new requirements and continuously improve our product using Scrum methodology. We handled new needs such as the interceptor, pet size, and type selection, and enhanced pop-up design, demonstrating our commitment to iterative improvement and customer satisfaction.

By considering these issues, we have ensured our project's compliance with legal and ethical standards and enhanced its social value and professional quality. We believe these considerations have improved our project's overall quality and professionalism as a development team.

Conclusion

Learning outcome:

Our project allowed us to explore various aspects of software development, from planning and design, implementation, testing, and debugging, to change management and consideration of legal, social, ethical, and professional issues. The journey gave us invaluable insights and lessons, which we will carry forward to our future endeavors.

One significant learning from the project was the critical role of effective communication and teamwork in the Scrum framework. We learned that to make Agile methodology work, and a team must foster a collaborative and transparent culture. We recognized that each team member's unique perspective added value to the project, strengthening our collective decision-making and problem-solving capabilities.

Understanding and responding to changes was another key learning from this project. By implementing the interceptor, pet size, and type selection and enhanced pop-up design, we learned how Scrum provides a flexible and adaptive structure to manage changes effectively.

This experience underlined the importance of maintaining a responsive and adaptable approach in software development.

Finally, the project emphasized the importance of considering legal, social, ethical, and professional issues in software development. Whether it was the encryption of passwords to ensure data privacy or the commitment to upholding intellectual property rights, these considerations deeply influenced our project decisions. They contributed to a product that is respectful of user privacy, legally robust, and of high quality.

Improvement in the future:

Looking towards future projects, there are areas where we could further improve.

Firstly, the planning phase can be better streamlined to prevent unnecessary revisions later in the project. Improved planning also helps define the scope more accurately and avoid scope creep.

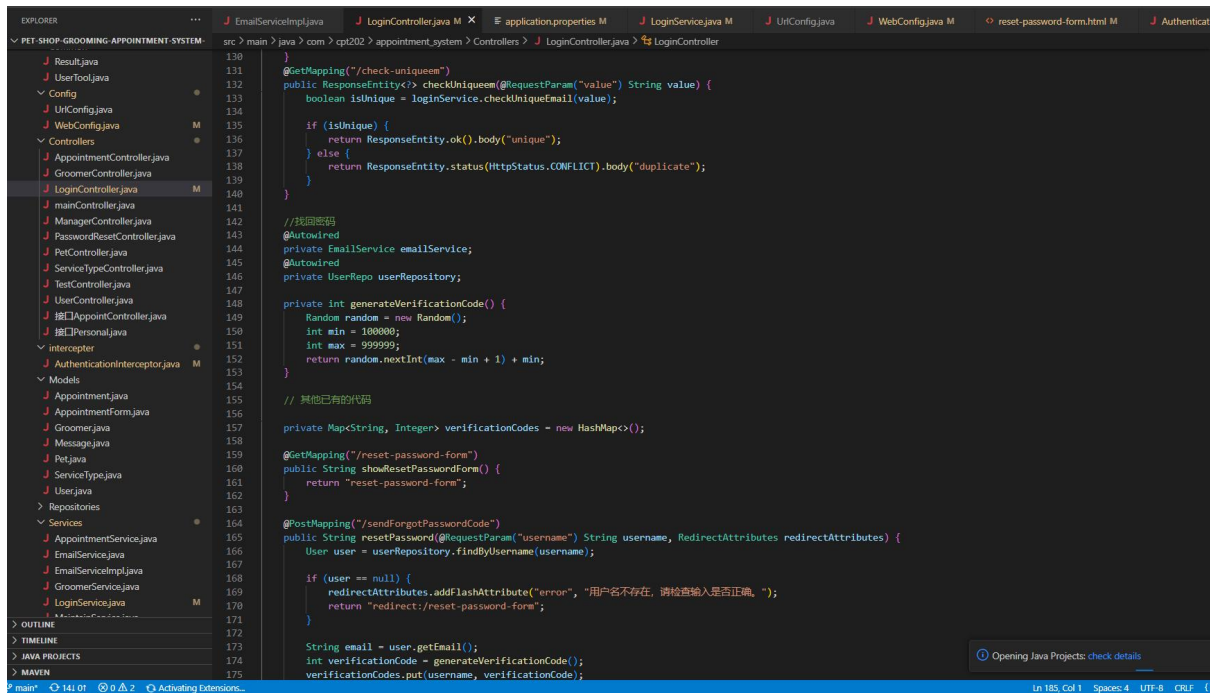
Secondly, we could focus more on automated testing to ensure all functionalities are validated efficiently and comprehensively. This step would help detect and fix issues more quickly and reduce the risks associated with human error in testing.

Thirdly, An area of improvement for future projects involves stakeholder communication and management. In this academic project, we did not interact with actual stakeholders, limiting our exposure to managing client expectations and gathering user feedback. As we move forward and engage in real-world projects, the ability to effectively communicate with stakeholders, understand their requirements, manage their expectations, and translate their feedback into actionable improvements will be essential. This skill set will be invaluable, and I look forward to developing it in my future projects.

In conclusion, this project served as an incredible learning opportunity, teaching us about the intricacies of software development and the importance of team collaboration, adaptability, and ethical considerations. As a computer science student, this project marked a significant milestone for me. Allowing me to apply the knowledge and skills I've accumulated over the past three years to create a comprehensive project from scratch. I had the opportunity to collaborate, discuss, and exchange ideas with a talented and knowledgeable team. This experience was not only rewarding in the present context, but I am confident it will bear substantial significance in my future learning and professional endeavors.

Finally, I would like to express my deepest gratitude to all the teachers, teaching assistants, and team members who have offered their invaluable help and support throughout this project.

Appendix



```
138 }
139
140 @GetMapping("/check-uniqueem")
141 public ResponseEntity<> checkUniqueem(@RequestParam("value") String value) {
142     boolean isUnique = loginService.checkUniqueEmail(value);
143
144     if (isUnique) {
145         return ResponseEntity.ok().body("unique");
146     } else {
147         return ResponseEntity.status(HttpStatus.CONFLICT).body("duplicate");
148     }
149 }
150
151 //找回密码
152 @Autowired
153 private EmailService emailService;
154 @Autowired
155 private UserRepo userRepository;
156
157 private int generateVerificationCode() {
158     Random random = new Random();
159     int min = 100000;
160     int max = 999999;
161     return random.nextInt(max - min + 1) + min;
162 }
163
164 // 其他已有的代码
165 private Map<String, Integer> verificationCodes = new HashMap<>();
166
167 @GetMapping("/reset-password-form")
168 public String showResetPasswordForm() {
169     return "reset-password-form";
170 }
171
172 @PostMapping("/sendForgotPasswordCode")
173 public String resetPassword(@RequestParam("username") String username, RedirectAttributes redirectAttributes) {
174     User user = userRepository.findByUsername(username);
175
176     if (user == null) {
177         redirectAttributes.addFlashAttribute("error", "用户名不存在, 请检查输入是否正确。");
178         return "redirect:/reset-password-form";
179     }
180
181     String email = user.getEmail();
182     int verificationCode = generateVerificationCode();
183     verificationCodes.put(username, verificationCode);
184 }
```

